Proyecto de big data e inteligencia artificial

Fase 1

StockPulse

CURSO DE ESPECIALIZACIÓN EN

INTELIGENCIA ARTIFICIAL Y BIG DATA

MODALIDAD ONLINE

10/03/2026

Jimmy Chen
Joaquín Fernández
Sergio Romero
Deninson Tapia

INDICE

1. División de roles **3**
2. Resumen ejecutivo **4**
3. Descripción funcional del sistema **4**
4. Alcance definitivo (IN/OUT) **5**
5. Arquitectura del sistema (alto nivel) **6**
Diagrama de arquitectura 6
Descripción de componentes 6
Flujo de datos 8
Separación de responsabilidades 8
6. Tecnologías seleccionadas **8**
Stack tecnológico completo 8
Criterios de decisión 10
Dependencias e incompatibilidades conocidas 10
7. Modelo de base de datos **11**
8. Selección de datos / Selección del Dataset **11**
9. Ética del proyecto **12**
10. Backlog inicial **13**
11. Plan de trabajo **15**
Cronograma de trabajo 15
Hitos principales 15
Dependencias entre tareas 16
Reparto de tareas 16
Riesgos técnicos 17
12. Estrategia de desarrollo **17**
Flujo de trabajo con Git 18
Integración del código 18
Validación de funcionalidad 19
Estrategia de testing 19
13. Riesgos técnicos **19**

1. División de roles

Jimmy Chen – Business Intelligence

Responsable de la definición del proyecto, alcance y funcionalidad del sistema desde el punto de vista del usuario.
Responsabilidades:
1. Título del proyecto e integrantes del equipo.
2. Resumen ejecutivo: explicación del problema, la solución propuesta y el valor del proyecto.
3. Descripción funcional del sistema: definición de lo que hace la aplicación y cómo interactúan los usuarios con ella.
4. Alcance definitivo (IN / OUT): delimitación de funcionalidades incluidas y excluidas del proyecto.

Joaquín Fernández – Platform / Project Manager

Responsable de la gestión del entorno de desarrollo, organización del proyecto y control del repositorio. Principal coordinador del grupo y las tareas.
Responsabilidades:
10. Backlog inicial: definición de las primeras tareas del proyecto y priorización del desarrollo.
11. Plan de trabajo: organización de las tareas del proyecto, planificación temporal y seguimiento del progreso del equipo.
12. Estrategia de desarrollo: definición de la metodología de trabajo (iteraciones, entregas y control de versiones).
14. Repositorio del proyecto: gestión del repositorio, estructura del código y control de versiones.

Sergio Romero – Data Engineer

Responsable del diseño técnico del sistema y de la definición de la infraestructura de datos necesaria para el funcionamiento del proyecto.
Responsabilidades:
5. Arquitectura del sistema (alto nivel): diseño de la estructura general del sistema, incluyendo flujo de datos y componentes principales.
6. Tecnologías seleccionadas: elección y justificación de las herramientas y tecnologías utilizadas en el proyecto.

Deninson Tapia – Data Scientist

Responsable del análisis de datos, modelado y aspectos éticos relacionados con los datos.
Responsabilidades:
7. Modelo de datos: definición de las estructuras de datos utilizadas en el proyecto.
8. Datos: selección de datasets, descripción de variables y preparación de los datos para el análisis.
9. Ética, legalidad y accesibilidad: revisión de posibles riesgos relacionados con privacidad,

uso responsable de datos y accesibilidad.
13. Riesgos técnicos actualizados: identificación de posibles problemas técnicos del proyecto y propuestas de mitigación.

2. Resumen ejecutivo

El proyecto StockPulse aborda el problema de la mala planificación de la demanda en muchas empresas que provoca exceso de inventario o roturas de stock. Estas situaciones generan pérdidas económicas, aumento de costes operativos y dificultades para garantizar la disponibilidad de productos en momentos de alta demanda.

La solución que proponemos consiste en desarrollar un sistema de análisis y predicción de ventas basado en datos. El sistema utilizará datos históricos y/o simulados de ventas que serán procesados mediante un pipeline de datos para su limpieza y organización. A partir de esta información, se aplicará un modelo de predicción capaz de identificar patrones de comportamiento y estimar la demanda futura. Los resultados se mostrarán en un dashboard interactivo, donde los usuarios podrán visualizar predicciones, comparar ventas reales con estimaciones y obtener información útil para la toma de decisiones.

El valor principal que ofrece StockPulse es mejorar la toma de decisiones en la gestión de inventario, permitiendo a las empresas anticiparse a cambios en la demanda, reducir pérdidas por sobrestock o falta de productos y optimizar la planificación de compras. Esto beneficia tanto a los responsables de compras y almacén como a la empresa en general, al mejorar la eficiencia y la disponibilidad de productos para los clientes.

3. Descripción funcional del sistema

Qué hace exactamente la aplicación

StockPulse es una aplicación de análisis de datos que permite predecir la demanda de productos utilizando datos históricos de ventas. El sistema procesa la información, aplica un modelo de predicción y muestra los resultados en un panel visual que ayuda a interpretar las tendencias y previsiones de ventas.

Flujo general de uso

1. El usuario carga datos de ventas desde archivos o datasets.
2. El sistema procesa y limpia los datos.
3. El modelo analiza los datos y genera predicciones de demanda.
4. El usuario visualiza los resultados en un dashboard con gráficos y métricas.

Casos principales de usuario

- Responsable de compras que consulta las predicciones para planificar pedidos.
- Responsable de almacén que analiza tendencias de ventas para gestionar el stock.
- Analista o responsable financiero que revisa datos para apoyar decisiones estratégicas.

Qué puede hacer el sistema

- Analizar datos de ventas.
- Detectar patrones de comportamiento.
- Generar predicciones de demanda.
- Mostrar resultados en un dashboard interactivo.

Qué no puede hacer el sistema

- Automatizar las compras de la empresa.
- Crear o gestionar una aplicación móvil.
- Utilizar modelos avanzados de deep learning dentro del alcance inicial del proyecto.

4. Alcance definitivo (IN/OUT)

Funcionalidades mínimas obligatorias

- **Carga de datos de ventas**: el sistema permitirá importar datos desde archivos o datasets públicos que contengan información como fecha, producto, unidades vendidas y precio.
- **Procesamiento y limpieza de datos**: los datos que se carguen pasarán por un proceso de preparación que incluirá la eliminación de valores duplicados, el manejo de valores vacíos y la organización de datos por fechas para facilitar el análisis.
- **Almacenamiento en base de datos**: los datos procesados se almacenarán en una base de datos estructurada para facilitar la consulta y análisis posterior.
- **Análisis de datos históricos**: el sistema analizará la evolución de las ventas a lo largo del tiempo para identificar patrones como estacionalidad o tendencias.
- **Modelo de predicción de demanda**: se implementará un modelo de análisis que permita estimar la demanda futura a partir de los datos históricos disponibles.
- **Generación de predicciones**: el sistema calculará estimaciones de ventas futuras que puedan servir de apoyo para la planificación de inventario.
- **Visualización de resultados**: se desarrollará un dashboard interactivo que mostrará gráficos, métricas y comparaciones entre ventas reales y predicciones para facilitar la interpretación de los resultados por parte de los usuarios.

Funcionalidades opcionales

- **Incorporación de variables externas** como festivos o factores de estacionalidad para mejorar la precisión del modelo de predicción.
- **Inclusión de indicadores visuales** en el dashboard que ayuden a detectar posibles riesgos de falta de stock o cambios en la demanda.
- **Comparación detallada** entre diferentes periodos de ventas para analizar tendencias.
- **Pruebas con distintos modelos de predicción** para evaluar cuál ofrece mejores resultados.
- **Mejora de la presentación** de datos mediante gráficos adicionales o métricas más detalladas en el dashboard.

Elementos descartados explícitamente

- **Desarrollo de una aplicación móvil**: el sistema se centrará en un entorno de análisis y visualización accesible desde ordenador.
- **Automatización de compras o reposición de inventario**: el objetivo del sistema es apoyar la toma de decisiones y no ejecutar acciones automáticas.

- **Uso de modelos avanzados de deep learning**: el proyecto se enfoca en modelos más simples y adecuados al alcance y tiempo disponible.
- **Integración con sistemas empresariales reales** como TPV: el proyecto utilizará datos exportados o datasets públicos.
- **Gestión completa del inventario**: la herramienta se centrará únicamente en el análisis y predicción de la demanda.

5. Arquitectura del sistema (alto nivel)

La arquitectura de StockPulse está organizada en capas funcionales claramente diferenciadas: ingesta, procesamiento, almacenamiento, modelado y visualización. El flujo de datos es lineal y progresivo: los datos entran en bruto, se limpian, se almacenan, se modelan y finalmente se presentan al usuario en el dashboard. A continuación se detalla cada componente.

Diagrama de arquitectura

El sistema sigue un pipeline de datos de extremo a extremo estructurado en las siguientes capas:

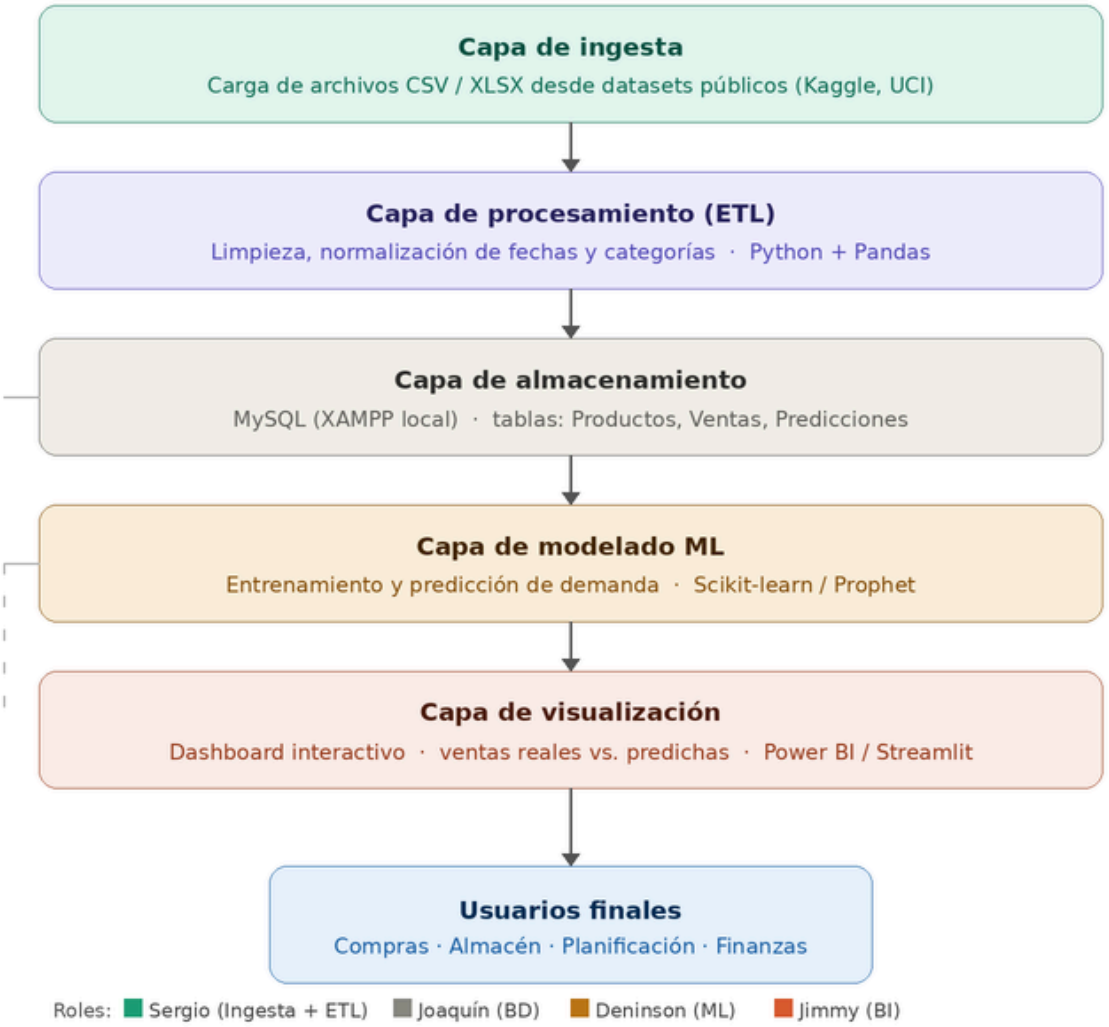
- Capa de ingesta: carga de archivos CSV/XLSX desde datasets públicos (Kaggle).
- Capa de procesamiento: limpieza, normalización y transformación mediante scripts Python.
- Capa de almacenamiento: base de datos MySQL para persistir los datos procesados.
- Capa de modelado: módulo de predicción de demanda (Scikit-learn / Prophet).
- Capa de visualización: dashboard interactivo (Power BI / Streamlit) conectado a la base de datos.

Descripción de componentes

Componente	Función principal	Tecnología prevista
Ingesta de datos	Carga de archivos CSV/XLSX desde dataset público (Kaggle)	Python + Pandas
Pipeline de limpieza	Eliminación de nulos, duplicados; normalización de fechas y categorías	Python + Pandas

Base de datos	Almacenamiento estructurado de ventas históricas y predicciones	MySQL (XAMPP local)
Modelo de predicción	Entrenamiento y generación de predicciones de demanda	Scikit-learn / Prophet
API backend (opcional)	Exposición de datos procesados y predicciones al dashboard	FastAPI (Python)
Dashboard / Visualización	Interfaz interactiva con gráficos, métricas y comparativas	Power BI / Streamlit

Nota: la API backend (FastAPI) es un componente opcional que se incorporará si la integración directa entre la base de datos y el dashboard lo requiere. En una primera iteración, el dashboard podrá conectarse directamente a MySQL.



Flujo de datos

El flujo de datos dentro del sistema sigue la siguiente secuencia:

- 1. El Data Engineer carga el dataset de ventas (CSV/XLSX) en el script de ingesta.
- 2. El pipeline de limpieza (Python + Pandas) elimina nulos, duplicados y normaliza fechas y categorías.
- 3. Los datos limpios se insertan en la base de datos MySQL (tablas: Productos, Ventas).
- 4. El módulo de predicción lee los datos históricos desde MySQL, entrena el modelo y escribe las predicciones en la tabla Predicciones.
- 5. El dashboard consulta MySQL y muestra los resultados: ventas reales vs. predichas, tendencias y alertas de bajo stock.

Separación de responsabilidades

Cada capa es independiente y puede modificarse sin afectar al resto, lo que facilita el trabajo en paralelo del equipo:

- Data Engineer (Sergio): ingesta y pipeline de limpieza.
- Data Analyst (Deninson): modelado y evaluación del modelo.
- Platform (Joaquín): base de datos, integración y repositorio.
- BI (Jimmy): dashboard y KPIs.

6. Tecnologías seleccionadas

La selección de tecnologías se ha basado en tres criterios principales: familiaridad del equipo, adecuación al alcance del proyecto y compatibilidad entre herramientas. Se ha evitado deliberadamente introducir complejidad innecesaria (por ejemplo, herramientas de orquestación como Airflow o frameworks de deep learning) para garantizar que el proyecto sea ejecutable dentro del tiempo disponible.

Stack tecnológico completo

Tecnología	Categoría	Justificación	Alternativa descartada
Python	Lenguaje principal	Ecosistema maduro para IA/Data. Todo el equipo tiene experiencia base y es el estándar del sector.	R (curva de aprendizaje mayor para el equipo)
Pandas	Librería de datos	Permite manipular y limpiar DataFrames de forma eficiente con sintaxis sencilla y bien documentada.	Polars (menos familiar para el equipo)
Scikit-learn	ML / Modelado	Librería estándar para regresión, evaluación de modelos y pipelines. Suficiente para el alcance del proyecto.	TensorFlow/Keras (excesivo para modelos simples; deep learning está fuera del alcance)

Prophet	ML (fallback)	Modelo de series temporales de Meta, robusto ante estacionalidad y festivos. Se usará si Scikit-learn no alcanza la precisión requerida.	ARIMA (más complejo de configurar)
MySQL	Base de datos	Base de datos relacional adecuada para datos tabulares estructurados (ventas, productos, predicciones). Bien integrada con Python vía mysql-connector.	PostgreSQL (válido, pero el equipo tiene más experiencia con MySQL)
XAMPP	Entorno local	Simplifica el despliegue de MySQL en local durante el desarrollo sin necesidad de configuración de servidor.	Docker (más robusto pero mayor complejidad de configuración inicial)
Power BI / Streamlit	Dashboard	Power BI para visualizaciones rápidas y profesionales; Streamlit como alternativa en Python si se prefiere un dashboard integrado en el mismo stack.	Tableau (requiere licencia de pago)
Git + GitHub	Control de versiones	Flujo de trabajo colaborativo estándar. Repositorio ya creado en la Fase 0.	GitLab (innecesario dado que ya se inició en GitHub)
Jupyter Notebook	Exploración y EDA	Entorno interactivo ideal para análisis exploratorio, visualización rápida y documentación del proceso.	Scripts Python puros (menos legibles para EDA)

Criterios de decisión

Las elecciones tecnológicas responden a los siguientes principios:

- Familiaridad del equipo: se prioriza el uso de herramientas que los integrantes ya conocen o han utilizado en otras asignaturas del curso, para reducir la curva de aprendizaje y maximizar la velocidad de desarrollo.
- Coste cero: todas las herramientas seleccionadas son gratuitas u open-source, eliminando cualquier barrera económica.

- Integración con Python: el núcleo del proyecto es Python, por lo que todas las herramientas deben integrarse de forma nativa o sencilla con este ecosistema.
- Escalabilidad controlada: las tecnologías elegidas permiten escalar el proyecto si el alcance crece (p.ej., pasar de MySQL local a un servidor, o de Streamlit a una app web completa), sin necesidad de reescribir el código.

Dependencias e incompatibilidades conocidas

Se han identificado las siguientes consideraciones técnicas que el equipo debe tener en cuenta durante el desarrollo:

- Python 3.10+ recomendado: algunas librerías como Prophet y versiones recientes de Scikit-learn requieren Python 3.9 o superior. Se documentará la versión exacta en el archivo environment/ del repositorio (requirements.txt o environment.yml).
- XAMPP solo para desarrollo local: MySQL en XAMPP no está preparado para producción. Si el proyecto necesita desplegarse para demostraciones externas, se migrará a un servidor MySQL independiente o a un servicio cloud gratuito (p.ej., Railway o PlanetScale).
- Power BI vs. Streamlit: ambas opciones están actualmente sobre la mesa. La decisión final se tomará en la Fase 2 en función del tiempo disponible y de la necesidad de interactividad dinámica. Streamlit ofrece mayor integración con Python; Power BI, una presentación más profesional con menor esfuerzo de desarrollo.
- Compatibilidad Prophet: la instalación de Prophet puede requerir compiladores C++ en Windows. Se recomienda usar un entorno virtual (venv o conda) para aislar las dependencias.

7. Modelo de base de datos

Implementaremos una base de datos MySQL en un entorno local usando XAMPP, para poder almacenar la base de datos de manera interna, para poder hacer consultas o analizar los datos del dataset de manera rápida y eficiente.

Modelo temporal (abierto a cambios)

Tenemos unos campos principales para almacenar los datos de los productos, las ventas relacionadas con el producto, y las predicciones. Las predicciones están relacionadas a un producto y una fecha.

Tabla Productos:

- id_producto | INT PK

- nombre | VARCHAR
- categoria | VARCHAR
- precio_unitario | DECIMAL

Tabla ventas:
- id_producto | INT FK
- id_venta | INT PK
- fecha_venta | DATE
- nombre | VARCHAR
- unidades_vendidas | INT
- total_venta | DECIMAL

Tabla predicciones:
- id_prediccion| INT PK
- id_producto | INT FK
- fecha_prediccion| DATE
- unidades_predichas | INT
- unidades_vendidas | INT
- confianza_modelo | DECIMAL

8. Selección de datos / Selección del Dataset

Usaremos como prueba y partida de inicio, una base de datos pública que hemos seleccionado en kaggle (Retail Sales Dataset - Sample Transactions) que contiene información de las transacciones de ventas de productos con una fecha respectiva, cantidad, etc.
El siguiente paso será convertir el archivo xlsx en csv, hacer una limpieza profunda de los datos para eliminar valores nulos, duplicados…, convertir las fechas en un formato estandar para la facilitación de agrupación de ventas de productos por fechas y normalizar nombres y categorías de productos para su posterior análisis.

Objetivo: Obtener una base de datos limpia y estructurada, para que sirva como insumo confiable para que nuestro futuro modelo de predicción de demanda y el dashboard de visualización sea lo más exacto posible.

9. Ética del proyecto

En un principio, al estar usando una base de datos pública, no se verá comprometida ninguna información personal, pero es importante considerar ciertos aspectos éticos y legales relacionados con el uso de estos a futuro.

- Protección de datos: El proyecto no está basado en datos personales de clientes o información significativa, esto reduce considerablemente los riesgos relacionados con la privacidad, por no decir que son inexistentes. Si a futuro se llega a desarrollar y se usan datos privados, deberíamos aplicar la normativa de protección de datos aplicables, como la RGPD en la UE.
- Uso responsable de los datos: Es importante tener en cuenta que el proyecto, el análisis y las predicciones no deben sustituir completamente el criterio de un profesional en el ámbito, es decir, tener el criterio humano. Una vez se obtengan los datos con un previo análisis, debe ser supervisado y tomar decisiones en base a los resultados y el criterio de un equipo o persona.
- Transparencia del modelo: La comprensión debe ser muy sencilla para una persona, deben ser simples y comprensibles para que se comprenda mejor y tomen un resultado conforme a lo estudiado y confíen en el sistema.

10. Backlog inicial

El backlog inicial define las primeras historias de usuario necesarias para construir una primera versión funcional del sistema StockPulse. Estas historias se priorizan según la dependencia técnica del pipeline de datos y el valor funcional del sistema.

ID	Historia de usuario	Descripción	Criterios de aceptación
HU1	Carga de dataset	Carga del dataset de ventas desde un archivo CSV para poder iniciar el proceso de análisis.	- El sistema acepta archivos CSV/XLSX. - Los datos se cargan correctamente en Python. - Se valida que el archivo contiene las columnas necesarias.
HU2	Limpieza de datos	Limpieza de los datos eliminando nulos y duplicados para asegurar la calidad del dataset.	- Se eliminan registros duplicados. - Se gestionan valores nulos. - Se normalizan formatos de fecha.
HU3	Normalización de datos	Transformación de los datos a un formato consistente para facilitar el análisis posterior.	- Las fechas están en formato estándar. - Los nombres de productos y categorías están normalizados. - Los tipos de datos son correctos.
HU4	Creación de base de datos	Creación de la base de datos MySQL con las tablas necesarias para almacenar los datos procesados.	- Se crean las tablas Productos, Ventas y Predicciones. - Las claves primarias y foráneas funcionan correctamente. - La base de datos acepta inserciones desde Python.
HU5	Inserción de datos en base de datos	Almacenamiento de los datos limpios en MySQL para que puedan ser utilizados por el modelo de predicción.	- Los datos se insertan correctamente en las tablas. - No hay pérdida de información. - Se pueden realizar consultas básicas.

HU6	Análisis exploratorio de datos (EDA)	Analizar los datos históricos para identificar patrones y tendencias de ventas.	- Se generan gráficos básicos de ventas. - Se identifican posibles tendencias o estacionalidades. - El análisis queda documentado en Jupyter Notebook.
HU7	Entrenamiento del modelo de predicción	Entrenamiento de un modelo de predicción para estimar la demanda futura de productos.	- El modelo se entrena con datos históricos. - Se generan predicciones para fechas futuras. - Se evalúa el rendimiento del modelo.
HU8	Generación de predicciones	Guardado de las predicciones generadas en la base de datos para poder visualizarlas posteriormente.	- Las predicciones se guardan en la tabla Predicciones. - Cada predicción está asociada a producto y fecha.
HU9	Conexión del dashboard con la base de datos	Conexión del dashboard a la base de datos para visualizar los resultados del análisis.	- El dashboard puede consultar datos desde MySQL. - Se muestran ventas reales y predicciones.
HU10	Visualización de métricas y gráficos	Visualización de gráficos de ventas y predicciones para interpretar los resultados fácilmente.	- El dashboard muestra gráficos de tendencias. - Se comparan ventas reales y predicciones. - Se visualizan métricas clave de ventas.

11. Plan de trabajo

El desarrollo del proyecto se organizó en una planificación semanal durante una semana de trabajo colaborativo. Las tareas se gestionaron mediante un

tablero de Trello, donde se asignaron responsabilidades y se realizó el seguimiento del progreso.

Cronograma de trabajo

Día	Actividad principal	Nº de tareas
Lunes	Planificación inicial del proyecto y asignación de tareas	1
Jueves	Desarrollo de arquitectura, tecnologías y definición técnica	4
Sábado	Desarrollo de modelo de datos, dataset y ética del proyecto	4
Domingo	Finalización del documento y revisión general	3
Domingo (final)	Ajustes finales, revisión y entrega	2

Total de tareas realizadas: 14 tareas

Hitos principales

Hito	Descripción
Definición del proyecto	Elección del problema, solución propuesta y alcance del sistema.
Diseño de arquitectura	Definición del pipeline de datos y componentes del sistema.
Selección tecnológica	Elección del stack tecnológico del proyecto.
Diseño de base de datos	Definición de las tablas y relaciones.
Selección y preparación de datos	Elección del dataset y definición del proceso de limpieza.

Documentación final	Revisión, corrección y preparación del documento final.

Dependencias entre tareas

Algunas tareas dependen de otras para poder desarrollarse correctamente:

Tarea	Depende de
Limpieza de datos	Carga del dataset
Entrenamiento del modelo	Datos limpios
Predicciones	Modelo entrenado
Dashboard	Base de datos y predicciones
Análisis de datos	Dataset cargado

Reparto de tareas

Integrante	Rol	Principales tareas
Jimmy Chen	Business Intelligence	Definición del proyecto, funcionalidades, dashboard y KPIs
Joaquín Fernández	Platform / Project Manager	Gestión del repositorio, backlog, planificación y estrategia de desarrollo
Sergio Romero	Data Engineer	Arquitectura del sistema, pipeline de datos e infraestructura
Deninson Tapia	Data Scientist	Dataset, modelo de datos, análisis y aspectos éticos

Riesgos técnicos

Riesgo	Impacto	Mitigación
Datos incompletos o inconsistentes	Predicciones poco precisas	Proceso riguroso de limpieza y validación
Baja precisión del modelo	Resultados poco útiles	Evaluar varios modelos (Scikit-learn, Prophet)
Problemas de integración	Retrasos en el desarrollo	Definir arquitectura clara y realizar pruebas parciales
Falta de tiempo	Trabajo incompleto	Organización mediante Trello y reparto claro de tareas

12. Estrategia de desarrollo

El desarrollo del proyecto StockPulse sigue una metodología iterativa, permitiendo avanzar progresivamente desde la ingesta de datos hasta la visualización final de resultados.

Flujo de trabajo con Git

El proyecto utiliza Git y GitHub para el control de versiones y colaboración entre los miembros del equipo.

Estructura de ramas:

- main
  Contiene la versión estable del proyecto.
- develop
  Rama principal de desarrollo donde se integran las nuevas funcionalidades.
- feature/
  Ramas utilizadas para desarrollar funcionalidades específicas.

Flujo de trabajo:

1. Cada miembro crea una rama feature.
2. Se desarrolla la funcionalidad correspondiente.
3. Se realiza un pull request hacia develop.
4. Tras revisión, se integra el código.
5. Cuando el sistema es estable, se fusiona con main.

Integración del código

La integración del sistema se realizará de forma progresiva siguiendo el pipeline de datos del proyecto:

1. Integración del pipeline de datos (ingesta y limpieza).
2. Integración con la base de datos MySQL.
3. Integración del modelo de predicción.
4. Integración con el dashboard de visualización.

Cada componente se probará de forma independiente antes de integrarse en el sistema completo.

Validación de funcionalidad

Para validar el correcto funcionamiento del sistema se realizarán pruebas básicas en cada etapa del pipeline:

- Verificación de carga correcta del dataset.
- Validación del proceso de limpieza de datos.
- Comprobación de inserción en la base de datos.
- Evaluación del modelo de predicción.
- Verificación de la conexión del dashboard con la base de datos.

Estrategia de testing

Se realizarán pruebas básicas para garantizar la estabilidad del sistema:

Testing de datos

- Validación de formatos.
- Control de valores nulos.
- Verificación de consistencia en fechas y categorías.

Testing del modelo

- Evaluación mediante métricas de error (MAE o RMSE).
- Comparación entre ventas reales y predicciones.

Testing de visualización

- Verificación de consultas desde el dashboard.
- Comprobación de gráficos y métricas mostradas.

13. Riesgos técnicos

Identificamos los principales riesgos técnicos que podrían existir a lo largo del proyecto, podrían surgir diferentes riesgos en un futuro que podrían afectar al funcionamiento del sistema pero por ahora identificamos los principales:

- **Calidad de los datos:** Uno de los pasos más importantes, se debe limpiar bien los datos antes de usarlos para eliminar valores nulos, tratar valores inconsistentes, outliers, etc. Todo enfocado para que el modelo sea lo más preciso posible con menos sesgos.
- **Precisión del modelo**: Puede que un solo modelo nos demuestre cierta precisión, pero convendría probar más modelos con tal de ver cual es el más eficaz y acertado para nuestra idea de proyecto.
- **Limitación de la base de datos:** En principio tenemos todos los datos que requerimos para llevar a cabo estudios temporales para analizar la demanda de mercado en ciertas zonas, un gran riesgo que conlleva no tener una columna necesaria sería la limitación del análisis.
- **Conexión e integración entre componentes del sistema:** Debemos armonizar todos archivos, programas y herramientas que usaremos para llevar a cabo nuestro análisis, un problema sería la incompatibilidad entre herramientas o la dificultad de poder hacer conexiones estables entre estas. Se debe definir una arquitectura clara e ir realizando integraciones y pruebas claras a lo largo del proyecto.
- **Tiempo y organización**: Cada integrante del grupo tiene sus tareas y proyectos, un grave problema sería la falta de comunicación, tiempo y ganas por parte de algún integrante si no llega a comunicarse correctamente, para esto tenemos organizado un calendario y tareas divididas con tal de llevar cada uno a cabo una tarea asignada dentro del tiempo que mejor le convenga.

Enlace del repositorio:
https://github.com/Joaquin-AI-DEV/Proyecto_IA_BigData_Grupo5